

セキュリティ実践演習第一 第2回 マルウェア解析

理工学部理工学科

知能情報システムプログラム

DX人材育成基盤プログラム

池部実 minoru@oita-u.ac.jp

マルウェア解析についての演習

- マルウェア解析(1) 表層解析
- マルウェア解析(2) 逆アセンブリ
- マルウェア解析(3) 動的解析
- マルウェア解析(4) コードからの特徴抽出
- マルウェア解析(5) 機械学習・深層学習によるマルウェア分類
- マルウェア対策・攻撃検知

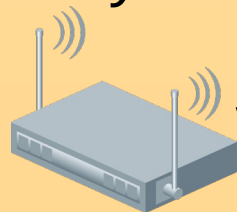
演習内容

- 機械学習などを用いてマルウェアを分類
 - 特徴量抽出
- 演習を踏まえて、マルウェア対策・攻撃検知のための方法を検討する

演習環境(1)

example.jp.
10.0.0.0/24

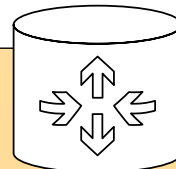
SSID: SecurityLab
(hidden)



Ubuntu
DHCP
10.0.0.0/24



ルータ(NAPT)



binary.example.jp.
10.0.0.160/24
(Ubuntu Server)

準備(1)

- 演習端末(DELL, Ubuntu)
 - login id: security
 - password: *****(Moodle上資料では非公開)
 - 端末に記載
 - 無線LAN: SecurityLab 接続(済)
- 演習用NWは, B-Coreで演習端末からのみ接続可
- 時間外に演習用サーバに接続する場合, 踏み台サーバを利用する(後述)

準備(2)

- サーバには, Ubuntu(ノートPC)のVSCodeから `ssh(extension)`で接続
- 接続先サーバ: `binary.example.jp`.
- サーバには, `security` ユーザでログイン
 - パスワードはUbuntu(ノートPC)と同じ
- `binary.example.jp`上で実行したウィンドウを手元のUbuntu(ノート)で表示したい場合,
`ssh -X binary.example.jp`
`ssh -Y binary.example.jp`
としてSSH接続する

~/ .ssh/config

- binary.example.jpで実行したmatplotlibのウィンドウをUbuntu(ノートPC)上に表示
- ~/ .ssh/configに以下のように記述

```
Host binary.example.jp
  HostName binary.example.jp
  User security
  ForwardX11 yes
  ForwardX11Trusted yes
```

(注意事項)Ubuntu上でのpip

- UbuntuなどのLinuxや, macOSでは共通環境へのpipインストールは推奨されていない
- ユーザ毎にvirtualenvの利用を推奨

```
python3 -m pip install matplotlib
error: externally-managed-environment
```

```
× This environment is externally managed
↳ To install Python packages system-wide, try apt install
python3-xyz, where xyz is the package you are trying to
install.
```

If you wish to install a non-Debian-packaged Python package, create a virtual environment using `python3 -m venv path/to/venv`. Then use `path/to/venv/bin/python` and `path/to/venv/bin/pip`. Make sure you have `python3-full` installed.

If you wish to install a non-Debian packaged Python application, it may be easiest to use `pipx install xyz`, which will manage a virtual environment for you. Make sure you have `pipx` installed.

See `/usr/share/doc/python3.12/README.venv` for more information.

note: If you believe this is a mistake, please contact your Python installation or OS distribution provider. You can override this, at the risk of breaking your Python installation or OS, by passing `--break-system-packages`.

hint: See PEP 668 for the detailed specification.

binary.example.jp

- sshでログインすると, venv環境を自動で読み込むように設定
 - (入力の必要なし) `source ~/.venv/bin/activate`
 - プロンプトの先頭に(.venv)が付いていればOK
- pipでインストールの際
 - `$ python3 -m pip MODULENAME`
- Binary Refineryはpipでインストール済み

演習内容

- `binary.example.jp:ml/MalwareDataset.csv`
vを分析して機械学習のための特徴量を抽出
- PEヘッダを用いた機械学習によるマルウェア
検出

バイナリ解析レポート

- 今回の演習では、`binary.example.jp`上でのターミナルでの実行結果をすべてテキストファイルに記載しておくこと

VSCodeでbinary.example.jp に接続

- ~/m1ディレクトリをオープン
- ターミナルを操作して、PythonのインタラクティブシェルでPythonのプログラムを実行していく

PE(Portable Executable)

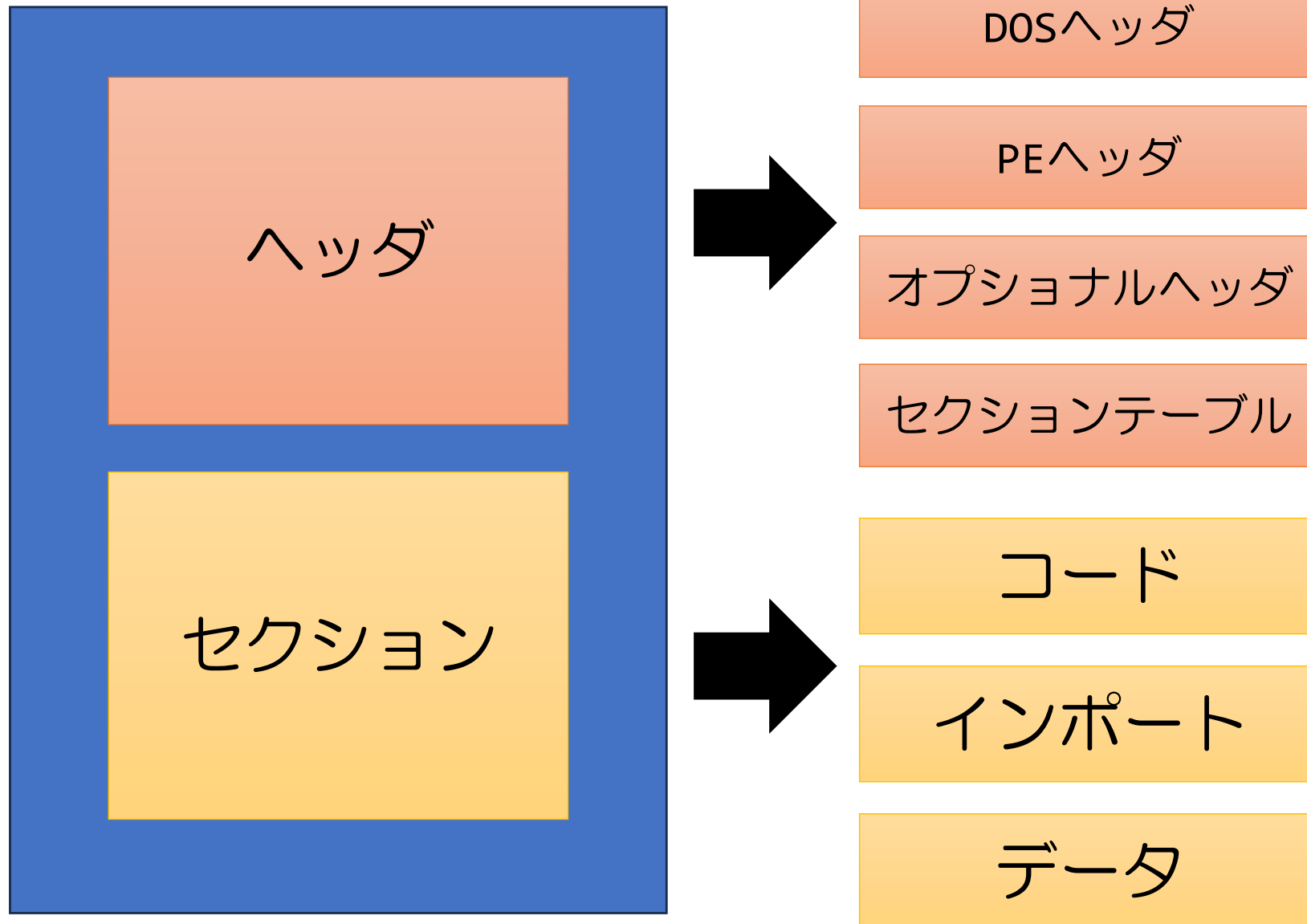
- PE(Portable Executable)形式
 - 32ビット/64ビットのWindowsで使用される実行可能ファイル, DLL(Dynamic Link Library), オブジェクトコードで採用されているファイル形式
 - インポート, エクスポート, タイムスタンプ, サブシステム, セクション, リソースなどマルウェア解析時に役立つ情報が含まれている
 - これらを特徴量としてマルウェア検出器を開発する

機械学習アルゴリズム

- ランダムフォレスト
 - 勾配ブースティング
 - AdaBoost
-
- それぞれどんな特徴がある機械学習アルゴリズムか調べてみよう

PEファイルの構造

PEファイル





図引用元:

<https://qiita.com/huzuni/items/824f5bcc1eb2c2949af1>

PEファイルの主なコンポーネント

- DOSヘッダ
 - すべてのPEファイルの先頭64バイトはDOSヘッダから開始
 - 古いOSであるMS-DOSは実行可能ファイルを検証し、DOSスタブモードで実行できる
- PEヘッダ
 - コードのサイズなどの情報を含む領域
- PEセクション
 - ファイルの主たるコンテンツを含む領域
- PEファイルのヘッダ情報確認
 - Peview, PeStduioなどのWindowsツールを利用
 - Pythonでは、pefileライブラリ

マルウェアデータセット

- Prateek Lalwani氏(セキュリティブロガー)によって配布されているマルウェアデータセットを利用して分析
- データセットに含まれる特徴量
 - 41323ファイルの正規Windowsバイナリ(.exe, .dll)から得られた特徴量
 - VirusShareからダウンロードできた96724検体のマルウェアから得られた特徴量
- ※様々なサイトでマルウェアの検体もダウンロード可能であるが、専用の端末で感染の危険のない環境(closed network)で取り扱うこと

不正指令電磁的記録に関する罪(いわゆるコンピュータ・ウイルスに関する罪)

- <https://www.keishicho.metro.tokyo.lg.jp/kurashi/cyber/law/virus.html>
- いわゆるコンピュータ・ウイルスの作成, 提供, 供用, 取得, 保管行為が罰せられる
 - 保管行為等にも正当な目的が必要なので, 勝手に学内ネットワーク内でマルウェアデータセットをダウンロードしないこと
 - pcap等のデータセットでもマルウェアが含まれるとFWで拒否されます

マルウェアデータセット

- Prateek Lalwani氏(セキュリティブロガー)によって配布されているマルウェアデータセットを利用して分析
- データセットに含まれる特徴量
 - 41323ファイルの正規Windowsバイナリ(.exe, .dll)から得られた特徴量
 - VirusShareからダウンロードできた96724検体のマルウェアから得られた特徴量
- 探索的データ解析
 - データセットの特徴を確認する準備
 - EDA: Exploratory Data Analysis

探索的データ解析

- 探索的データ解析
 - データの集計, 要約, 可視化といった処理により, データの特徴や異常を把握
 - 解析の結果から, 全体のアプローチを決めたり, 機械学習の出発点を見つける
- Pythonのydata-profilingで探索的データ解析が可能
 - 旧パッケージ名: pandas-profiling
 - ただし, Numpyのバージョンの制約があるので, `binary.example.jp`上では動かない
- Pandasやmatplotlib, SweetVizでデータ分析していく

インタラクティブシェル上の Pythonプログラム記述例

```
$ python3
```

```
Python 3.12.3 (main, Nov 6 2025, 13:44:16) [GCC 13.3.0] on linux
```

```
Type "help", "copyright", "credits" or "license" for more information.
```

```
>>> 大なり記号(>>>)の後に、Pythonのプログラムを1行ずつ記述(行末でエンター)
```

```
(出力結果がある場合・ない場合がある)
```

```
## for文などで、インデントする場合 ... のあとに、タブ文字(■)を挿入
```

```
>>> for i in range(3):
```

```
... ■ print('hoge')
```

```
...
```

```
hoge
```

```
hoge
```

```
hoge
```

```
インタラクティブシェルを抜ける際は、Ctrl-D
```

Pandasでcsvファイルを読み込む (青字を入力)

```
>>> import pandas as pd
>>> MalwareDataset =
pd.read_csv('MalwareData.csv', sep='|')
>>> print(MalwareDataset)
```

	Name	md5	Machine	...	LoadConfigurationSize	VersionInformationSize	legitimate
0	memtest.exe	631ea355665f28d4707448e442fbf5b8	332	...	0	16	1
1	ose.exe	9d10f99a6712e28f8acd5641e3a7ea6b	332	...	72	18	1
2	setup.exe	4d92f518527353c0db88a70fddcfd390	332	...	72	18	1
3	DW20.EXE	a41e524f8d45f0074fd07805ff0c9b12	332	...	72	18	1
4	dwtrig20.exe	c87e561258f2f8650cef999bf643a731	332	...	72	18	1
...	...	...	...	...	...	...	...
138042	VirusShare_8e292b418568d6e7b87f2a32aee7074b	8e292b418568d6e7b87f2a32aee7074b	332	...	72	0	0
138043	VirusShare_260d9e2258aed4c8a3bbd703ec895822	260d9e2258aed4c8a3bbd703ec895822	332	...	0	15	0
138044	VirusShare_8d088a51b7d225c9f5d11d239791ec3f	8d088a51b7d225c9f5d11d239791ec3f	332	...	72	14	0
138045	VirusShare_4286dccf67ca220fe67635388229a9f3	4286dccf67ca220fe67635388229a9f3	332	...	0	0	0
138046	VirusShare_d7648eae45f09b3adb75127f43be6d11	d7648eae45f09b3adb75127f43be6d11	332	...	72	0	0

[138047 rows x 57 columns]

(※すべてのデータは表示されず、省略した形で出力)

このあと、続けてプログラムを実行するのでインタラクティブシェルは終了しないこと

(続)Pandasで要素を指定して出力

それぞれ何が出力されるか確認

```
>>> print(MalwareDataset['legitimate'])
```

```
>>> print(MalwareDataset[1:4])
```


Sweetvizによる探索的データ解析

```
>>> import sweetviz as sv
```

```
/home/security/.venv/lib/python3.12/site-  
packages/sweetviz/graph.py:8: UserWarning:  
pkg_resources is deprecated as an API. See  
https://setuptools.pypa.io/en/latest/pkg\_resources.html.  
The pkg_resources package is slated for removal as  
early as 2025-11-30. Refrain from using this package or  
pin to Setuptools<81.
```

```
    from pkg_resources import resource_filename
```

```
>>> report = sv.analyze(MalwareDataset)
```

```
Feature: SizeOfImage
```

```
| ██████████ ██████████ ██████████ ██████████ ██████████ ██████████ | [ 40%]
```

```
00:07 -> (00:07 left)
```

(続)Sweetvizによる探索的データ解析

Done! Use 'show' commands to display/save.



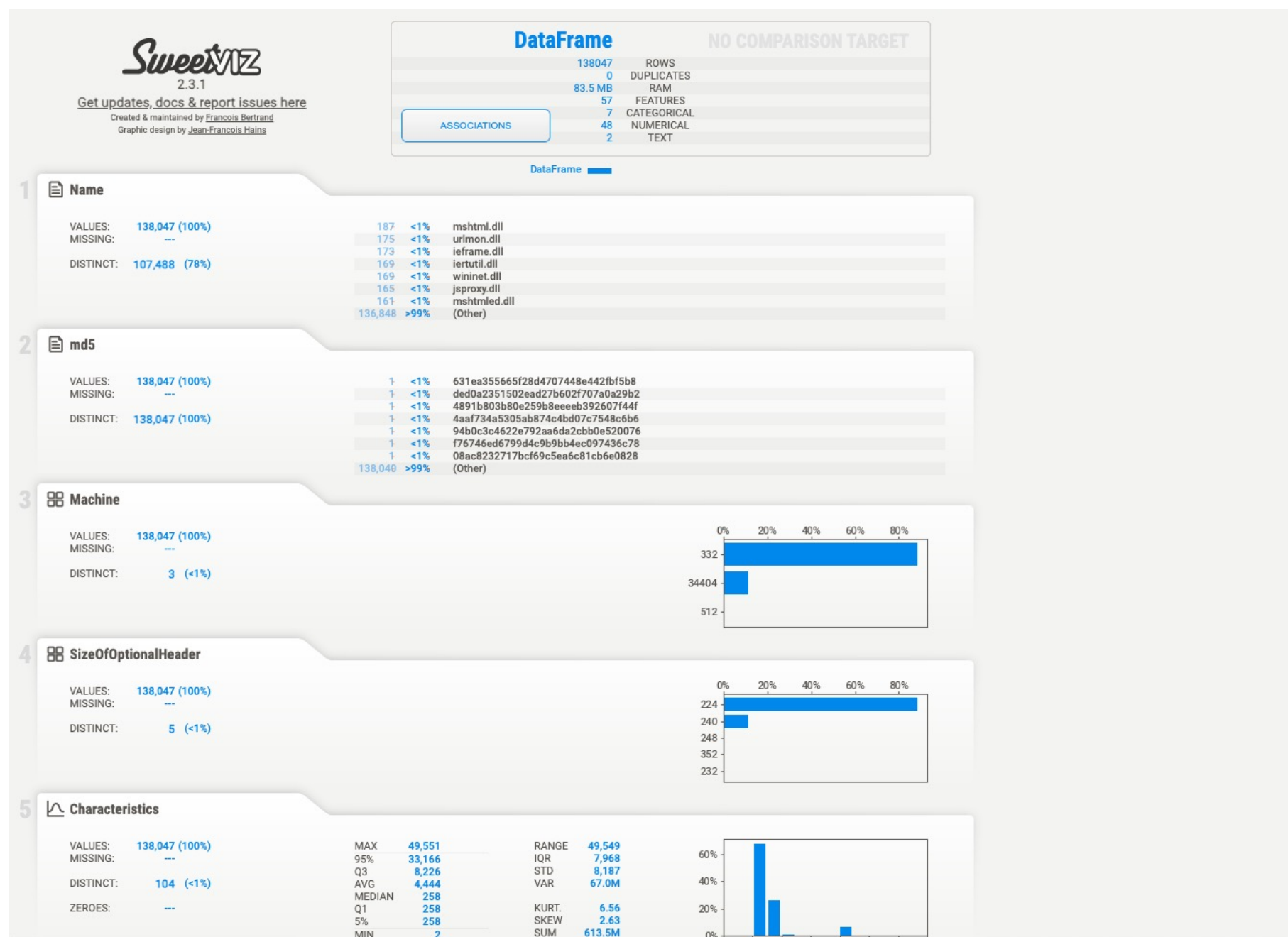
```
>>> report.show_html('malwaredataset.html')
```

VSCodeで直接HTMLファイルを開くか、scpで手元にコピーしてWebブラウザで開く

// Ubuntu(ノートPC)でscpを実行してコピー

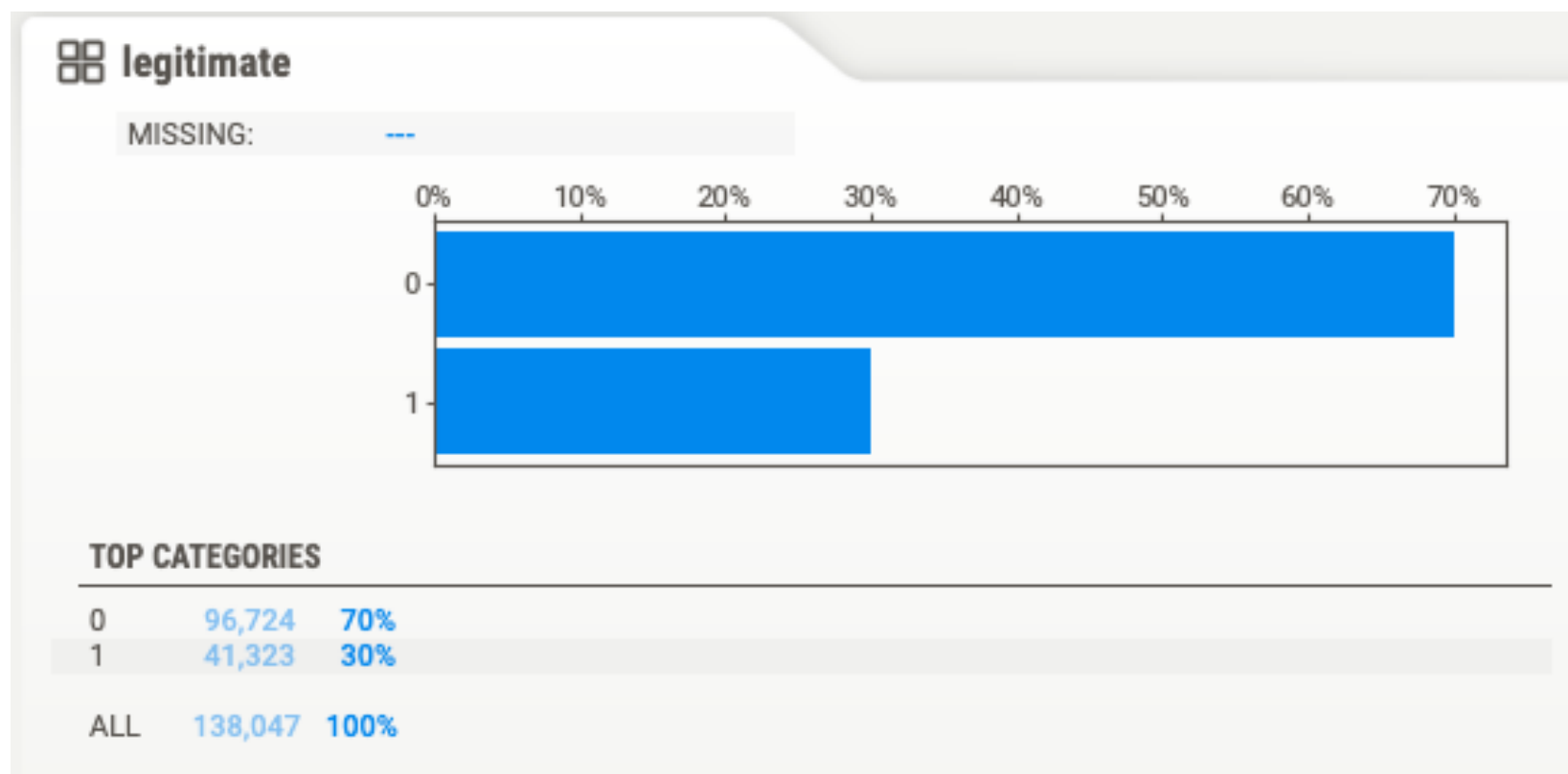
```
$ scp binary:m1/malwaredataset.html ./
```

MalwareDataset.csv分析結果

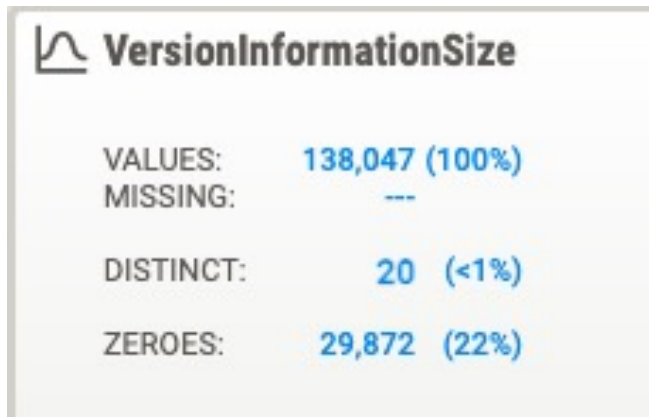


ラベル: legitimate

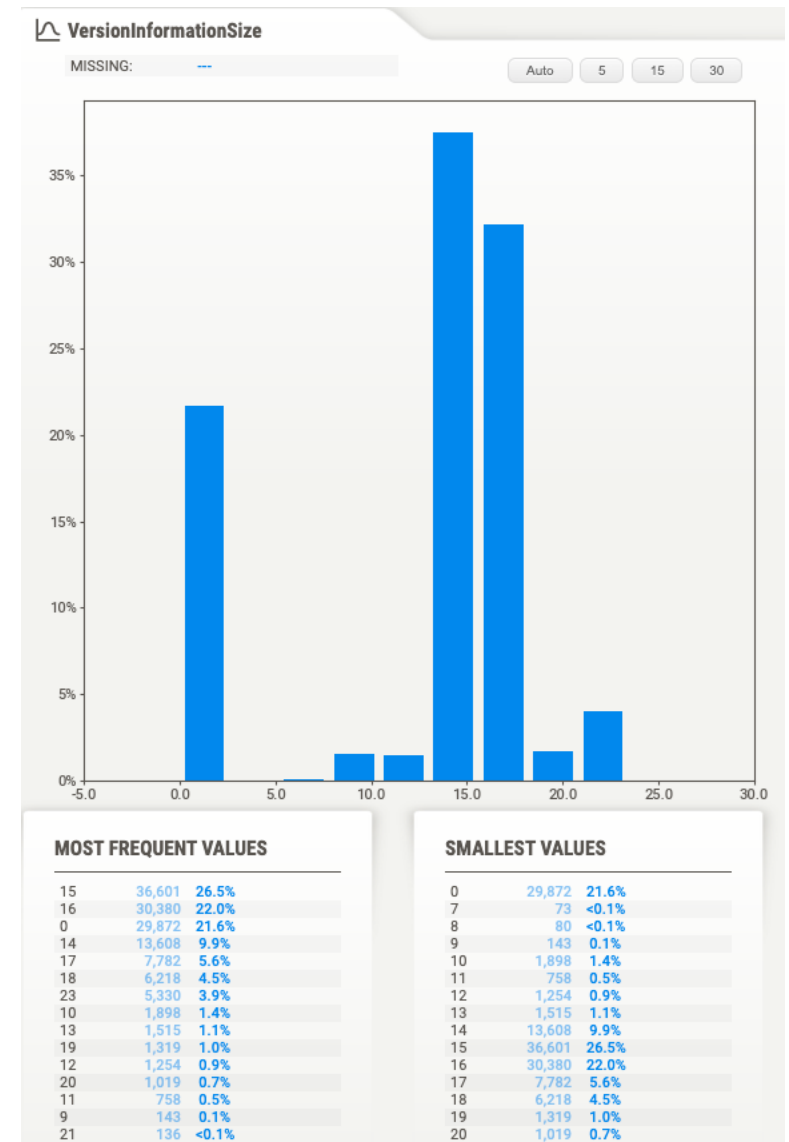
- 正規のWindowsバイナリかどうかを表すラベル
 - 1: 正規のWindowsバイナリ(41323件)
 - 0: マルウェアのバイナリ(96724件)



ラベル: VersionInformationSize



MAX	26.0	RANGE	26.0
95%	19.0	IQR	3.00
Q3	16.0	STD	6.80
MEDIAN	15.0	VAR	46.2
AVG	12.4		
Q1	13.0	KURT.	-0.316
5%	0.0	SKEW	-1.04
MIN	0.0	SUM	1.7M



ラベル: VersionInformationSize

- DISTINCT: 20
 - 20種類の値をとっている
 - うち, 0(Zeros)が22%
- MIN(最小値): 0
- MAX(最大値): 26.0
- AVG(平均値): 12.4

VersionInformationSize	
VALUES:	138,047 (100%)
MISSING:	---
DISTINCT:	20 (<1%)
ZEROES:	29,872 (22%)

MAX	26.0	RANGE	26.0
95%	19.0	IQR	3.00
Q3	16.0	STD	6.80
MEDIAN	15.0	VAR	46.2
AVG	12.4	KURT.	-0.316
Q1	13.0	SKEW	-1.04
5%	0.0	SUM	1.7M
MIN	0.0		

データ探索： 特徴量と目的変数の関係性確認

- 例) 特徴量のVersionInformationSizeと目的変数のlegitimateに関連性があるのかないのかを調べる
 - legitimateは、正規バイナリかマルウェアバイナリかで判定すべき値で今回の目的変数としている

データ探索(1): 特徴量と目的変数の関係性確認

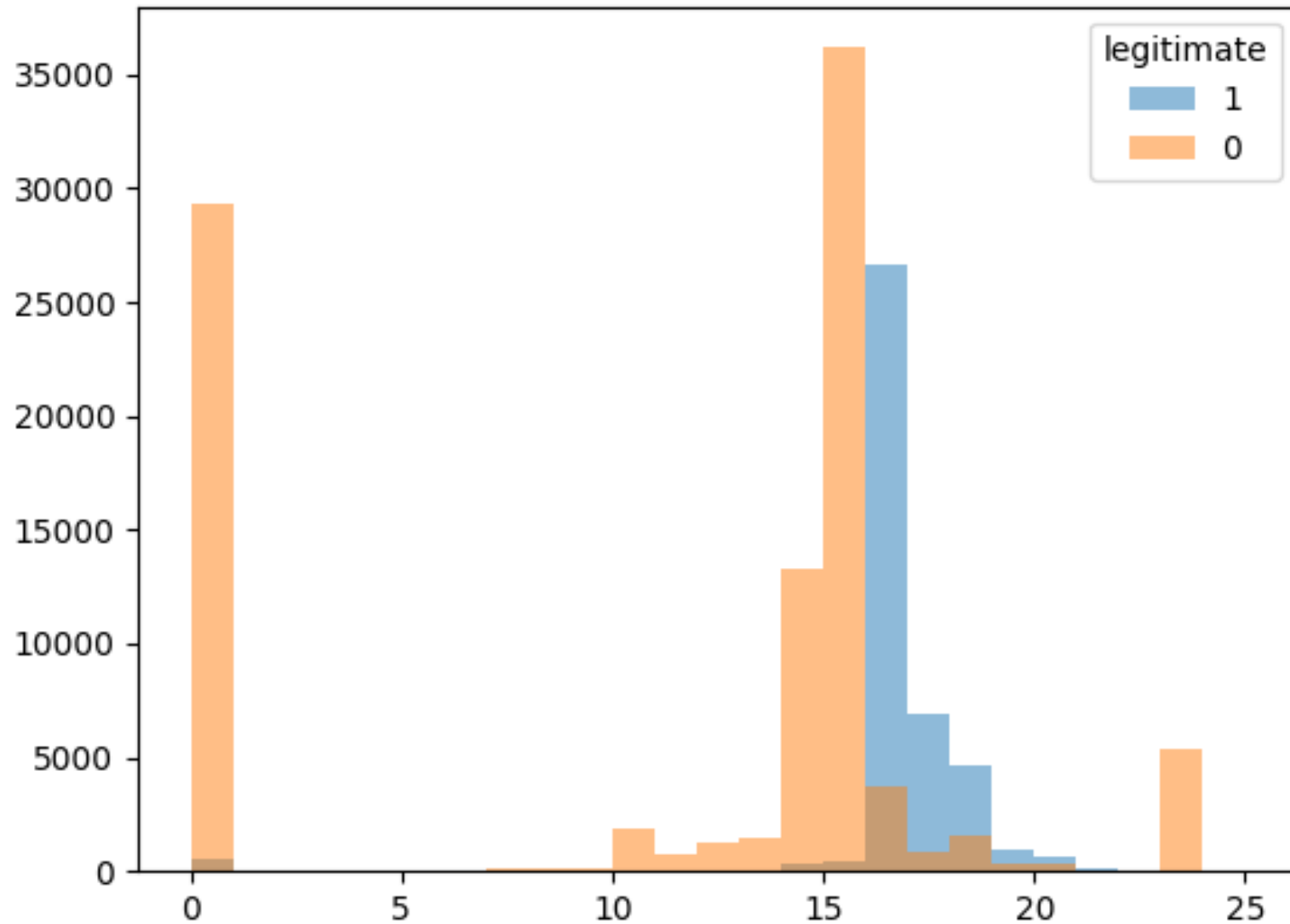
- VersionInformationSizeとlegitimateに
関係性があるかどうかを可視化して確認

```
>>> import matplotlib.pyplot as plt
>>>
plt.hist(MalwareDataset.loc[MalwareDataset['legitimate'] == 1, 'VersionInformationSize'], range(0,26),
alpha=0.5, label='1')

>>>
plt.hist(MalwareDataset.loc[MalwareDataset['legitimate'] == 0, 'VersionInformationSize'], range(0,26),
alpha=0.5, label='0')

>>> plt.legend(title='legitimate')
>>> plt.xlim(0,26)
>>> plt.savefig('legitimate_vis.png')
>>> plt.show()
```

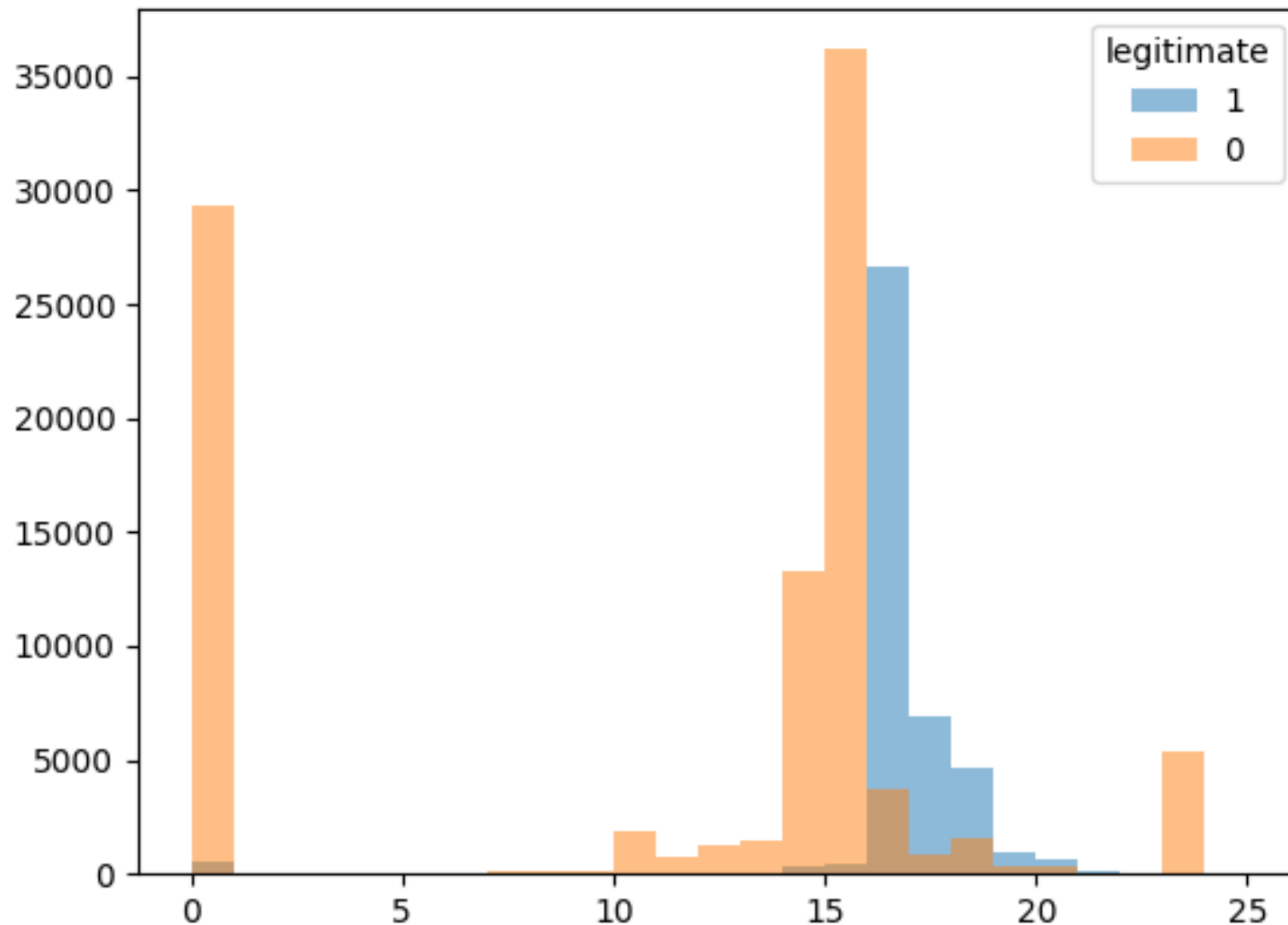

legitimate_vis.png
binaryから手元にscpでコピー



legitimate_vis.png binaryから手元にscpでコピー

このグラフから読み取れる仮説は？

オレンジ：マルウェア
青：正規バイナリ



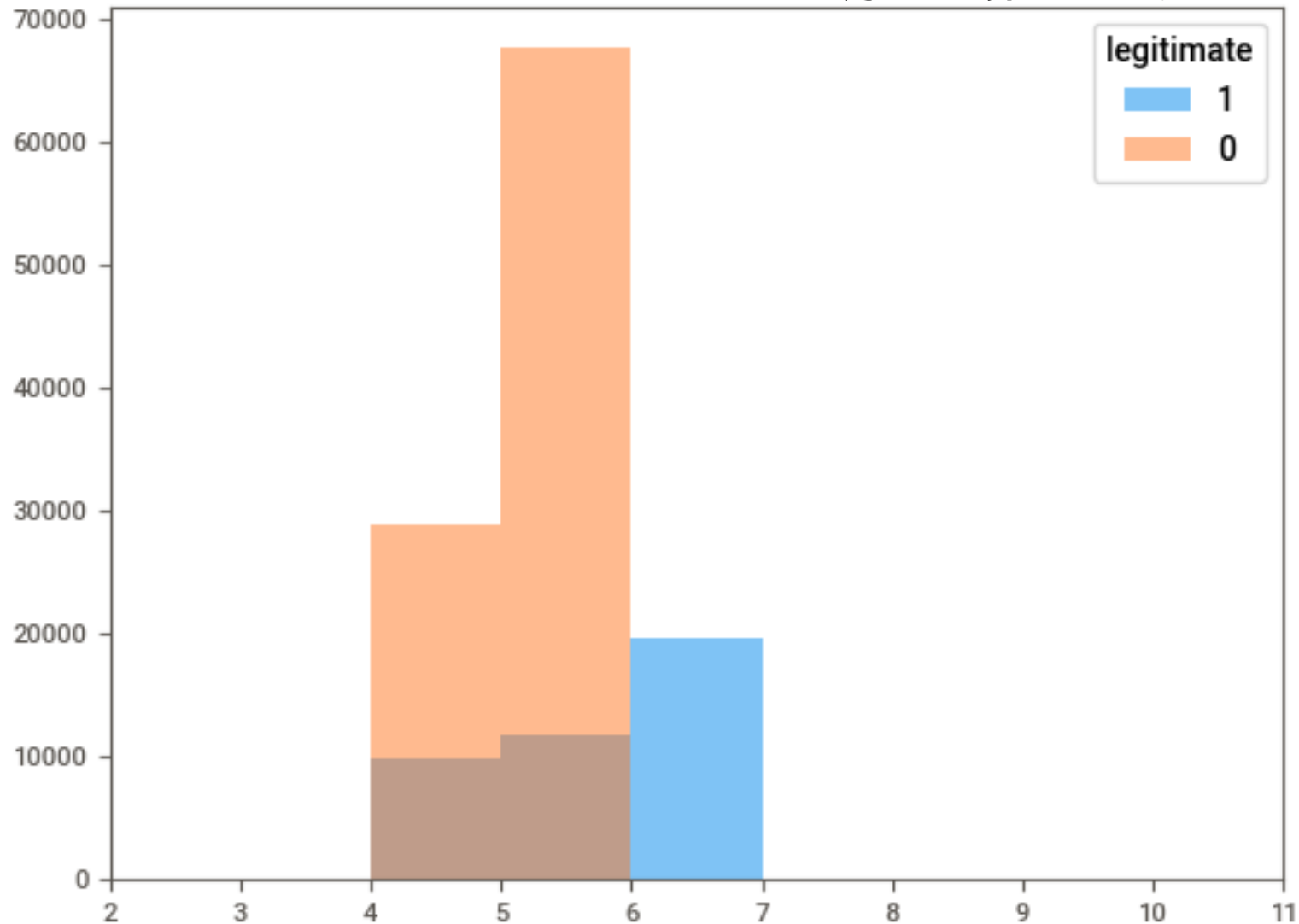
データ探索(2): 特徴量と目的変数の関係性確認

- MajorSubsystemVersionとlegitimateに
関係性があるかどうかを可視化して確認

```
>>> plt.gca().clear()
>>>
plt.hist(MalwareDataset.loc[MalwareDataset['legitimate'] == 1, 'MajorSubsystemVersion'], range(0,10),
alpha=0.5, label='1')
>>>
plt.hist(MalwareDataset.loc[MalwareDataset['legitimate'] == 0, 'MajorSubsystemVersion'], range(0,10),
alpha=0.5, label='0')
>>> plt.legend(title='legitimate')
>>> plt.xlim(0,26)
>>> plt.savefig('legitimate_msv.png')
>>> plt.show()
```

legitimate_vis.png
binaryから手元にscpでコピー

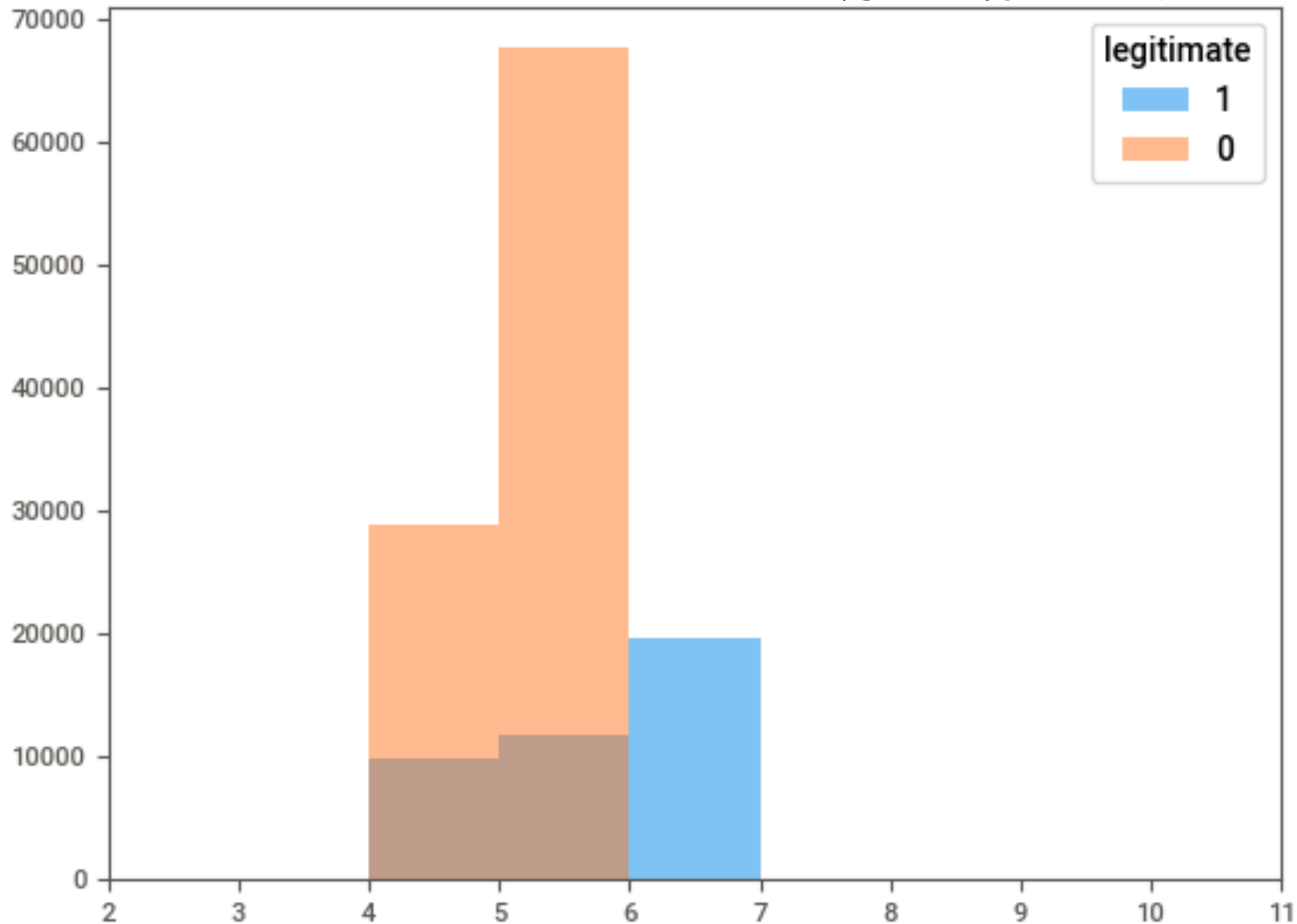
オレンジ: マルウェア
青: 正規バイナリ



legitimate_vis.png
binaryから手元にscpでコピー

このグラフから読み取れる仮説は？

オレンジ：マルウェア
青：正規バイナリ



機械学習アルゴリズムによる 特徴量の選択(半自動)

- 先ほどの例は可視化した結果を人が読み取り仮説をたてていた
- 機械学習アルゴリズムで結果を確認し、重要度の高い特徴量のみを選択する半自動的な手法を試す
- モデルの汎化性能への寄与が低い特徴量があると、性能の低下、メモリや計算リソースの消費を招いてしまう可能性がある
- 特徴選択により、有効な特徴量を残したまま全体の特徴量を減らすことができる

木構造の特徴選択

- ExtraTreesClassifierによる特徴量選択

- ~/ml/etc.pyを実行

```
$ cd ~/ml
```

```
$ python3 etc.py
```

木構造の特徴選択

Feature: SizeOfUninitializedData	Importance: 0.20134
Feature: Machine	Importance: 0.12239
Feature: Characteristics	Importance: 0.11797
Feature: SizeOfInitializedData	Importance: 0.10288
Feature: MajorLinkerVersion	Importance: 0.07446
Feature: SizeOfCode	Importance: 0.06810
Feature: SectionAlignment	Importance: 0.06664
Feature: ImageBase	Importance: 0.05752
Feature: BaseOfCode	Importance: 0.05308
Feature: BaseOfData	Importance: 0.04470
Feature: MinorLinkerVersion	Importance: 0.03135
Feature: SizeOfOptionalHeader	Importance: 0.03100
Feature: AddressOfEntryPoint	Importance: 0.02857

木構造の特徴選択

ジニ係数

Feature: SizeOfUninitializedData	Importance: 0.20134
Feature: Machine	Importance: 0.12239
Feature: Characteristics	Importance: 0.11797
Feature: SizeOfInitializedData	Importance: 0.10288
Feature: MajorLinkerVersion	Importance: 0.07446
Feature: SizeOfCode	Importance: 0.06810
Feature: SectionAlignment	Importance: 0.06664
Feature: ImageBase	Importance: 0.05752
Feature: BaseOfCode	Importance: 0.05308
Feature: BaseOfData	Importance: 0.04470
Feature: MinorLinkerVersion	Importance: 0.03135
Feature: SizeOfOptionalHeader	Importance: 0.03100
Feature: AddressOfEntryPoint	Importance: 0.02857

※ 重要な特徴量をジニ係数をもとにした重要度順に出力
ExtraTreesClassifierは毎回ランダムな木構造のため、
特徴量ごとの重要度の出力は変化する

ランダムフォレストの ハイパーパラメータチューニング

- ランダムフォレスト分類器を用いて, optuna を使ってハイパーパラメータをチューニング
- 探索範囲を少なくするため
 - 値がカテゴリをとるもの
 - 初期値が1ないし2で整数値
- ほかのパラメータをチューニング対象に入れると, 性能向上(時間や計算資源があれば)

ランダムフォレストの ハイパーパラメータチューニング

- RandomForestClassifier
- ~/ml/rfc.pyを実行

```
$ cd ~/ml
```

```
$ python3 rfc.py
```

ランダムフォレストの ハイパーパラメータチューニング

```
$ python3 rfc.py
```

```
[I 2026-01-08 21:12:40,915] A new study created in memory with name: no-name-c53a5399-88b9-4adf-bb9a-2d0e71238dc0
```

```
[I 2026-01-08 21:13:19,310] Trial 0 finished with value: 0.9949835703716003 and parameters: {'criterion': 'entropy', 'bootstrap': False, 'max_features': 'sqrt', 'min_samples_split': 5, 'min_samples_leaf': 1}. Best is trial 0 with value: 0.9949835703716003.
```

```
[I 2026-01-08 21:13:47,969] Trial 1 finished with value: 0.9947753038849513 and parameters: {'criterion': 'entropy', 'bootstrap': False, 'max_features': 'log2', 'min_samples_split': 4, 'min_samples_leaf': 2}. Best is trial 1 with value: 0.9947753038849513.
```

```
[I 2026-01-08 21:14:36,676] Trial 2 finished with value: 0.9933355646672479 and parameters: {'criterion': 'gini', 'bootstrap': False, 'max_features': 'sqrt', 'min_samples_split': 4, 'min_samples_leaf': 9}. Best is trial 2 with value: 0.9933355646672479.
```

```
[I 2026-01-08 21:14:58,328] Trial 3 finished with value: 0.9929914761858324 and parameters: {'criterion': 'entropy', 'bootstrap': True, 'max_features': 'log2', 'min_samples_split': 2, 'min_samples_leaf': 6}. Best is trial 3 with value: 0.9929914761858324.
```

```
[I 2026-01-08 21:15:35,582] Trial 4 finished with value: 0.993000533335896 and parameters: {'criterion': 'gini', 'bootstrap': True, 'max_features': 'sqrt', 'min_samples_split': 4, 'min_samples_leaf': 7}. Best is trial 3 with value: 0.9929914761858324.
```

```
[I 2026-01-08 21:15:57,378] Trial 5 finished with value: 0.9936977625293227 and parameters: {'criterion': 'entropy', 'bootstrap': True, 'max_features': 'log2', 'min_samples_split': 5, 'min_samples_leaf': 4}. Best is trial 3 with value: 0.9929914761858324.
```

```
params: {'criterion': 'entropy', 'bootstrap': True, 'max_features': 'log2', 'min_samples_split': 2, 'min_samples_leaf': 6}
```

```
Accuracy: 99.30098 %
```

```
[[19361  105]
```

```
 [  88 8056]]
```

ランダムフォレストによる 検出器の訓練結果

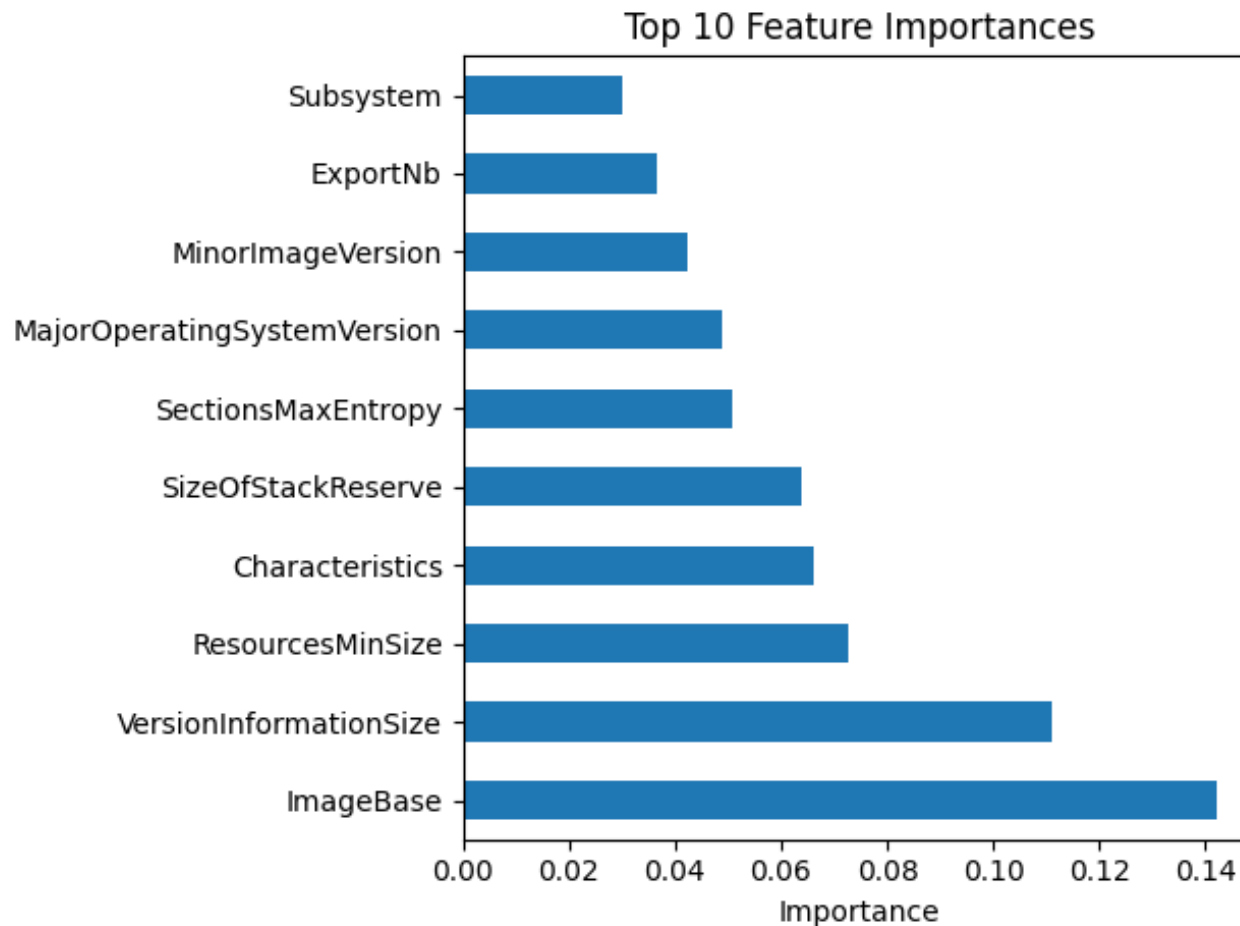
Accuracy: 99.30098 %

[[19361 105]

[88 8056]]

ランダムフォレストによる 検出器の訓練結果

- 特徴量の重要度の可視化結果
(RFC_feature_importances.png)



勾配ブースティングの ハイパーパラメータチューニング

- 機械学習アルゴリズムとして勾配ブースティングを用いる
- ハイパーパラメータをチューニング
 - チューニング対象のパラメータは探索幅を少なくするために、カテゴリをもつものを中心に選択

勾配ブースティングの ハイパーパラメータチューニング

- GradientBoostingClassifier

- ~/ml/gbc.pyを実行

```
$ cd ~/ml
```

```
$ python3 gbc.py
```


勾配ブースティングの ハイパーパラメータチューニング

```
$ python3 gbc.py
```

```
[I 2026-01-08 21:11:59,464] A new study created in memory with name: no-name-164c113e-0b63-40ad-8f01-de7afbba435b
```

```
[I 2026-01-08 21:12:10,510] Trial 0 finished with value: 0.9920318725099602 and parameters: {'max_depth': 10, 'max_features': 'log2', 'learning_rate': 0.11510689125059745, 'criterion': 'squared_error'}. Best is trial 0 with value: 0.9920318725099602.
```

```
[I 2026-01-08 21:12:15,143] Trial 1 finished with value: 0.9919956537486417 and parameters: {'max_depth': 6, 'max_features': 'log2', 'learning_rate': 0.1982134776344112, 'criterion': 'squared_error'}. Best is trial 1 with value: 0.9919956537486417.
```

```
[I 2026-01-08 21:12:22,268] Trial 2 finished with value: 0.9914885910901848 and parameters: {'max_depth': 7, 'max_features': 'sqrt', 'learning_rate': 0.32718918770602384, 'criterion': 'friedman_mse'}. Best is trial 2 with value: 0.9914885910901848.
```

```
[I 2026-01-08 21:12:28,084] Trial 3 finished with value: 0.9921767475552337 and parameters: {'max_depth': 7, 'max_features': 'log2', 'learning_rate': 0.38923933568429225, 'criterion': 'friedman_mse'}. Best is trial 2 with value: 0.9914885910901848.
```

```
[I 2026-01-08 21:12:31,456] Trial 4 finished with value: 0.9906917783411806 and parameters: {'max_depth': 4, 'max_features': 'log2', 'learning_rate': 0.4104463437029583, 'criterion': 'squared_error'}. Best is trial 4 with value: 0.9906917783411806.
```

```
params: {'max_depth': 4, 'max_features': 'log2', 'learning_rate': 0.4104463437029583, 'criterion': 'squared_error'}
```

```
Accuracy: 99.29011 %
```

```
[[19358 108]
```

```
[ 88 8056]]
```

勾配ブースティングによる 検出器の訓練結果

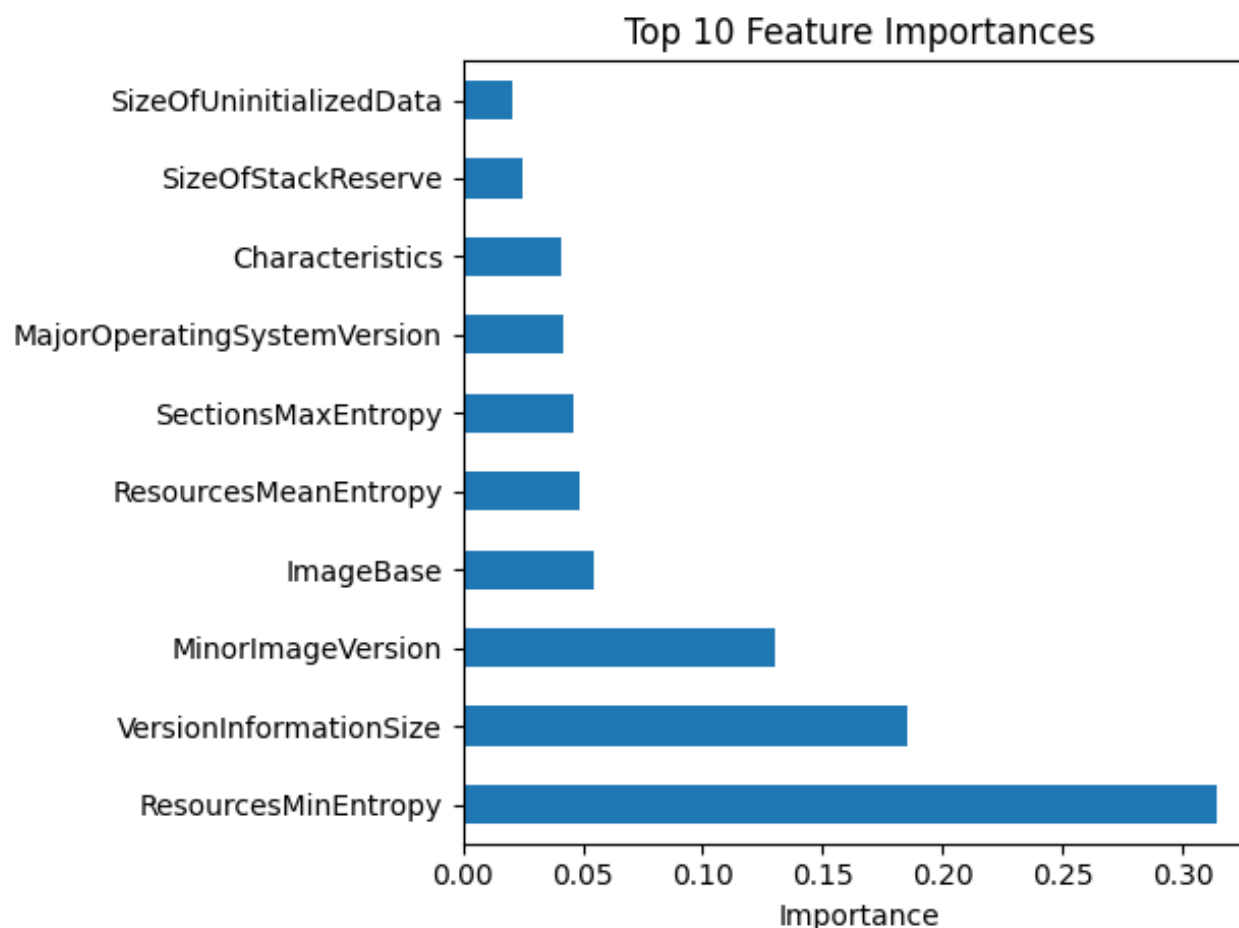
Accuracy: 99.29011 %

[[19358 108]

[88 8056]]

勾配ブースティングによる 検出器の訓練結果

- 特徴量の重要度の可視化結果
(GBC_feature_importances.png)



ImageBaseが
トップの予定だが...?

AdaBoostの ハイパーパラメータチューニング

- AdaBoostを用いて、ハイパーパラメータをチューニング
- 探索範囲を少なくするため
 - 値がカテゴリをとるもの
 - 初期値が1ないし2で整数値
- ほかのパラメータをチューニング対象に入れると、性能向上(時間や計算資源があれば)

AdaBoostの ハイパーパラメータチューニング

- AdaBoostClassifier
- ~/ml/abc.pyを実行

```
$ cd ~/ml
```

```
$ python3 abc.py
```

AdaBoostの ハイパーパラメータチューニング

```
$ python3 gbc.py
```

```
[I 2026-01-08 21:25:17,112] A new study created in memory with name: no-name-9a7942a6-59e7-48e4-9a88-bb7a8ea52ffb
```

```
[I 2026-01-08 21:25:36,218] Trial 0 finished with value: 0.9550059880581232 and parameters: {'algorithm': 'SAMME', 'learning_rate': 0.013028155503745292}. Best is trial 0 with value: 0.9550059880581232.
```

```
[I 2026-01-08 21:25:55,272] Trial 1 finished with value: 0.9860553911241684 and parameters: {'algorithm': 'SAMME', 'learning_rate': 0.4021153576989671}. Best is trial 0 with value: 0.9550059880581232.
```

```
[I 2026-01-08 21:26:14,145] Trial 2 finished with value: 0.9808125848085492 and parameters: {'algorithm': 'SAMME', 'learning_rate': 0.0411482062245152}. Best is trial 0 with value: 0.9550059880581232.
```

```
[I 2026-01-08 21:26:32,970] Trial 3 finished with value: 0.9836015003523897 and parameters: {'algorithm': 'SAMME', 'learning_rate': 0.16826635821499775}. Best is trial 0 with value: 0.9550059880581232.
```

```
params: {'algorithm': 'SAMME', 'learning_rate': 0.013028155503745292}
```

```
Accuracy: 95.35675 %
```

```
[[19201  265]
```

```
 [ 1017  7127]]
```

AdaBoostによる 検出器の訓練結果

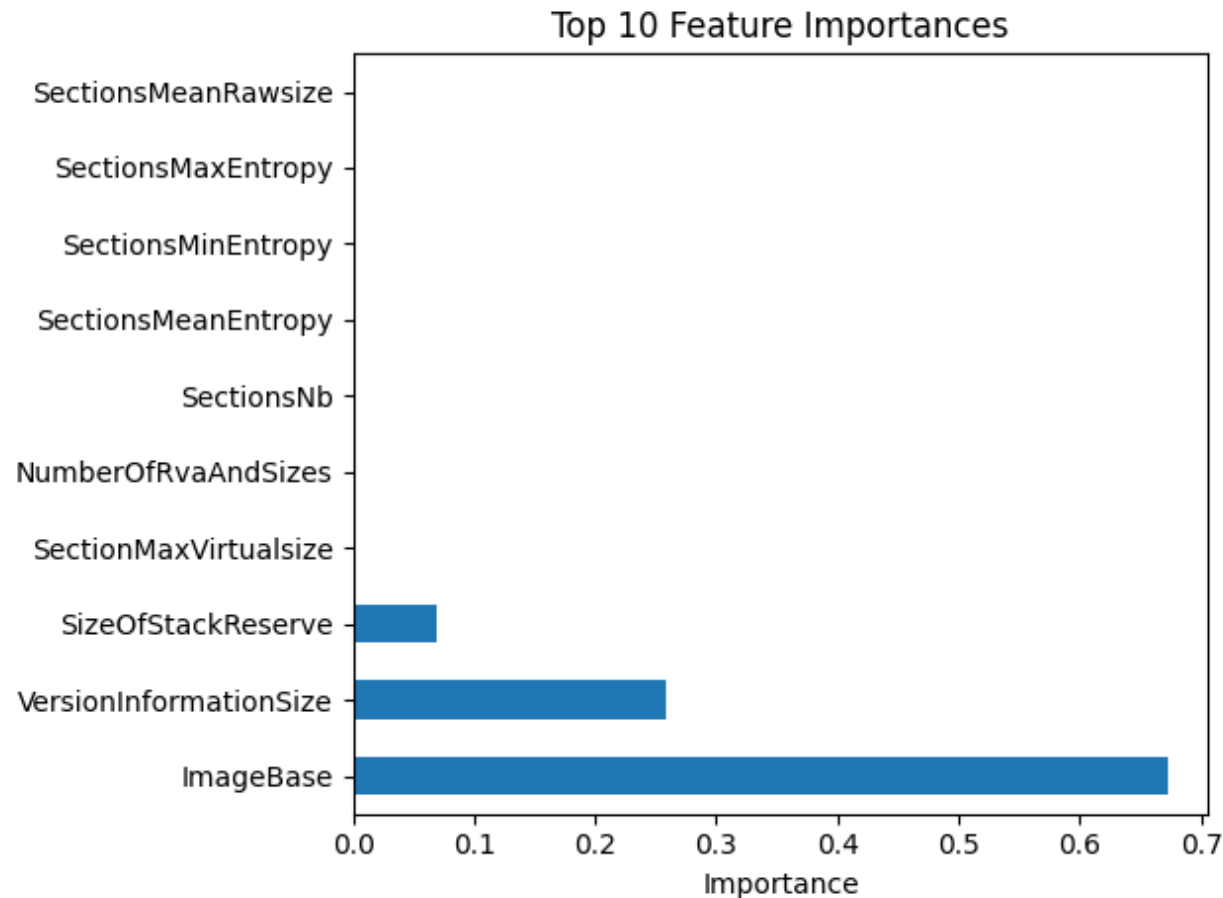
Accuracy: 95.35675 %

[[19201 265]

[1017 7127]]

AdaBoostによる 検出器の訓練結果

- 特徴量の重要度の可視化結果
(ABC_feature_importances.png)



3つの機械学習アルゴリズムの比較

- 特徴量の重要度は異なるものの, ImageBaseの特徴量としての役割は大きい
 - 勾配ブースティングはあまり？